

Lecture 23: Spark GraphX Programming

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

IIT Bhilai

April 15, 2026

Overview of GraphX

What is GraphX?

- GraphX is a component in Apache Spark for graphs and graph-parallel computation.
- It extends the Spark RDD abstraction by introducing the **Property Graph**.
- It provides a uniform API for both ETL (Extract, Transform, Load) and iterative graph computation.
- It unifies data-parallel and graph-parallel systems.

Key Features

- **Property Graph:** Directed multigraph with properties on vertices and edges.
- **Optimized Representation:** Efficiently stores and processes large-scale graphs.
- **Graph Operators:** Fundamental operations like subgraph, joinVertices, and aggregateMessages.
- **Pregel API:** A vertex-centric iterative computation model.
- **Built-in Algorithms:** PageRank, Connected Components, and Triangle Counting.

Graph-Parallel vs. Data-Parallel

- **Data-Parallel (Spark RDD):** Operations are independent across data elements.
- **Graph-Parallel (GraphX):** Operations on a vertex depend on its neighbors.
- GraphX enables switching between these views efficiently.

The Property Graph

Definition: Property Graph

- A **directed multigraph** with user-defined objects attached to each vertex and edge.
- **Directed:** Edges have a source and destination.
- **Multigraph:** Multiple parallel edges can exist between the same vertices.
- **VertexId:** Each vertex has a unique 64-bit long identifier.

Vertex Identifiers and Properties

- Vertices are keyed by a unique 64-bit long identifier (`VertexId`).
- GraphX does not impose any ordering constraints on vertex identifiers.
- Properties are stored as objects and can be any serializable type.
- Edges have corresponding source and destination vertex identifiers.

Property Graph Parameters

The property graph is parameterized over two types:

1. **VD (Vertex Data):** The type of object associated with each vertex.
2. **ED (Edge Data):** The type of object associated with each edge.

```
class Graph[VD, ED]
```

Memory Optimization

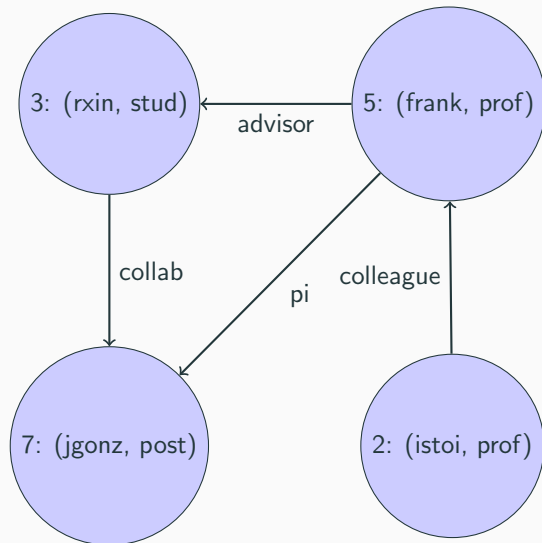
- GraphX optimizes the representation of vertex and edge types when they are **primitive data types** (e.g., int, double).
- This reduces the in-memory footprint by storing them in specialized arrays.
- Large-scale graphs benefit significantly from this optimization.

Modeling with Inheritance

In some cases, vertices might have different property types (e.g., a bipartite graph of users and products).

```
1 class VertexProperty()  
2 case class UserProperty(val name: String) extends VertexProperty  
3 case class ProductProperty(val name: String, val price: Double) extends VertexProperty  
4 // The graph might then have the type:  
5 var graph: Graph[VertexProperty, String] = null
```

Property Graph Illustration



- Property graphs are **immutable**, just like RDDs.
- Changes produce a new graph, but structural indices are reused for efficiency.
- Graphs are **distributed** across executors using vertex partitioning heuristics.
- Fault-tolerant via RDD lineage.

Constructing a Graph (Scala)

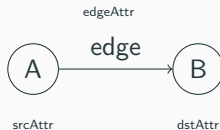
```
1 val users: RDD[(VertexId, (String, String))] =  
2   sc.parallelize(Seq((3L, ("rxin", "student")), (7L, ("jgonzal", "postdoc")),  
3                     (5L, ("franklin", "prof")), (2L, ("istoica", "prof"))))  
4 val relationships: RDD[Edge[String]] =  
5   sc.parallelize(Seq(Edge(3L, 7L, "collab"),    Edge(5L, 3L, "advisor"),  
6                     Edge(2L, 5L, "colleague"), Edge(5L, 7L, "pi")))  
7 val defaultUser = ("John Doe", "Missing")  
8 val graph = Graph(users, relationships, defaultUser)
```

Constructing a Graph (Python/GraphFrames)

```
1 from graphframes import GraphFrame
2 v = spark.createDataFrame([
3     ("3", "rxin", "student"), ("7", "jgonzal", "postdoc"),
4     ("5", "franklin", "prof"), ("2", "istoica", "prof")
5 ], ["id", "name", "pos"])
6 e = spark.createDataFrame([
7     ("3", "7", "collab"),      ("5", "3", "advisor"),
8     ("2", "5", "colleague"), ("5", "7", "pi")
9 ], ["src", "dst", "relationship"])
10 g = GraphFrame(v, e)
```

The Triplet View

- A **triplet** logically joins vertex and edge properties.
- Contains: `srcId`, `srcAttr`, `dstId`, `dstAttr`, and `attr` (edge attribute).
- Useful for computations that require properties of both endpoints.



Triplet SQL Analogy

The triplet join can be expressed as a SQL join:

```
1 SELECT src.id, dst.id, src.attr, e.attr, dst.attr
2 FROM edges AS e
3 LEFT JOIN vertices AS src ON e.srcId = src.id
4 LEFT JOIN vertices AS dst ON e.dstId = dst.id
```

Accessing Triplets (Scala)

```
1 val facts: RDD[String] = graph.triplets.map(triplet =>
2   triplet.srcAttr._1 + " is the " + triplet.attr + " of " + triplet.dstAttr._1
3 )
4 facts.collect.foreach(println)
```

Graph Operators

Operator Categories

GraphX operators are divided into:

1. **Property Operators:** Transform attributes.
2. **Structural Operators:** Change graph structure.
3. **Join Operators:** Combine graph with external data.
4. **Neighborhood Aggregation:** Compute aggregates over neighbors.

Property Operators

- `mapVertices`: Transforms vertex attributes.
- `mapEdges`: Transforms edge attributes.
- `mapTriplets`: Transforms edge attributes using vertex data.

Note: These do NOT change the graph structure (indices are reused).

mapVertices Details (Scala)

```
1 // Using VertexId and attribute in map
2 val newGraph = graph.mapVertices((id, attr) => mapUdf(id, attr))
```

It returns a new graph with the vertex properties modified by the user defined map function.

mapVertices Example (Python)

```
1 # Using GraphFrames
2 from pyspark.sql.functions import when, concat, lit
3 new_v = g.vertices.withColumn("name",
4     when(g.vertices.pos == "prof", concat(lit("Prof. "), g.vertices.name))
5     .otherwise(g.vertices.name))
6 new_g = GraphFrame(new_v, g.edges)
```

Structural Operators

- `reverse`: Reverses the direction of all edges.
- `subgraph`: Returns a graph with vertices and edges satisfying predicates.
- `mask`: Returns a graph containing only common vertices and edges.
- `groupEdges`: Merges parallel edges using a user function.

reverse Operator

```
1 def reverse: Graph[VD, ED]
```

- Returns a new graph with all edge directions reversed.
- Useful for computing inverse PageRank.
- Efficiently implemented without data movement or duplication.

subgraph Details

```
1 def subgraph(epred: EdgeTriplet[VD,ED] => Boolean = (x => true),  
2             vpred: (VertexId, VD) => Boolean = ((v, d) => true)): Graph[VD, ED]
```

- Takes vertex and edge predicates.
- Returns a graph containing only matching vertices and edges.
- Can be used to restrict the graph to interest areas.

subgraph Example (Scala)

```
1 // Keep only professors and their relationships
2 val profGraph = graph.subgraph(vpred = (id, attr) => attr._2 == "prof")
```

mask Operator

```
1 def mask[VD2, ED2](other: Graph[VD2, ED2]): Graph[VD, ED]
```

- Returns a graph that contains the vertices and edges that are also found in the input graph.
- Useful in conjunction with `subgraph` to restrict a graph based on another graph's properties.

groupEdges Operator

```
1 def groupEdges(merge: (ED, ED) => ED): Graph[VD, ED]
```

- Merges parallel edges (duplicate edges between pairs of vertices).
- Combined into a single edge reducing the size of the graph.
- In many numerical applications, weights can be added.

- `joinVertices`: Joins an RDD with vertices.
- `outerJoinVertices`: Similar to `joinVertices` but preserves vertices without matches.

```
def joinVertices[U](table: RDD[(VertexId, U)])(map: (VertexId, VD, U) => VD): Graph[VD, ED]
```

joinVertices Example (Scala)

```
1 val nonUniqueCosts: RDD[(VertexId, Double)] = ...
2 val uniqueCosts = graph.vertices.aggregateUsingIndex(nonUniqueCosts, _ + _)
3 val joinedGraph = graph.joinVertices(uniqueCosts)(
4   (id, oldCost, extraCost) => oldCost + extraCost)
```

outerJoinVertices Example

```
1 val outDegrees: VertexRDD[Int] = graph.outDegrees
2 val degreeGraph = graph.outerJoinVertices(outDegrees) {
3   (id, oldAttr, outDegOpt) => outDegOpt match {
4     case Some(outDeg) => outDeg
5     case None => 0
6   }
7 }
```


Neighborhood Aggregation

The Aggregation Problem

- Many graph algorithms require computing properties of neighbors.
- Examples:
 - Sum of follower ages.
 - Number of friends.
 - Smallest neighbor ID.
- Key operator: `aggregateMessages`.

- **sendMsg:** Function applied to each triplet. Can send a message to `src`, `dst`, or both.
- **mergeMsg:** Function to combine multiple messages at a vertex.
- **tripletFields:** (Optimization) Specifies which fields of the triplet are accessed.

aggregateMessages Signature

```
1 def aggregateMessages[Msg: ClassTag](  
2   sendMsg: EdgeContext[VD, ED, Msg] => Unit,  
3   mergeMsg: (Msg, Msg) => Msg,  
4   tripletFields: TripletFields = TripletFields.All)  
5 : VertexRDD[Msg]
```

Older Followers Example (Logic)

Goal: Compute the average age of the more senior followers of each user.

1. Map: For each edge, if `srcAge > dstAge`, send `(1, srcAge)` to destination.
2. Reduce: Sum counts and ages.
3. Finalize: Divide total age by count.

Older Followers Example (Scala)

```
1 val olderFollowers: VertexRDD[(Int, Double)] = graph.aggregateMessages[(Int, Double)](  
2   triplet => {  
3     if (triplet.srcAttr > triplet.dstAttr) {  
4       triplet.sendToDst((1, triplet.srcAttr))  
5     }  
6   },  
7   (a, b) => (a._1 + b._1, a._2 + b._2)  
8 )
```

Calculating Average Age

```
1 val avgAgeOfOlderFollowers: VertexRDD[Double] =  
2   olderFollowers.mapValues( (id, value) => value match {  
3     case (count, totalAge) => totalAge / count  
4   })
```

Optimization: TripletFields

- Specifies what data is accessed in the EdgeContext.
- Options: Src, Dst, All, None.
- Example: If only srcAttr is needed, use TripletFields.Src.
- Allows GraphX to select an optimized join strategy.

Computing Degree Information

- inDegrees: Number of inbound edges.
- outDegrees: Number of outbound edges.
- degrees: Total number of edges.

```
1 val maxInDegree: (VertexId, Int) = graph.inDegrees.reduce(max)
```

Collecting Neighbors

- `collectNeighborIds(edgeDirection)`
- `collectNeighbors(edgeDirection)`
- These can be costly as they duplicate information.
- Use `aggregateMessages` if possible for better performance.

Caching and Uncaching

Caching in GraphX

- RDDs are not persisted in memory by default.
- Graphs behave the same way.
- Use `graph.cache()` or `graph.persist()` when using a graph multiple times.
- For iterative computations, `graph.cache()` is essential.

- Intermediate results can fill up the cache.
- Uncaching may be necessary for best performance.
- It's difficult to unpersist graphs correctly because they are composed of multiple RDDs.
- **Recommendation:** Use the Pregel API which handles unpersisting intermediate results.

The Pregel API

Pregel: Iterative Computation

- A bulk-synchronous parallel (BSP) messaging model.
- Computation proceeds in **supersteps**.
- Vertices receive messages, update state, and send messages for the next step.
- Unlike standard Pregel, GraphX Pregel can send messages to any neighbor.

Pregel Parameters

- `initialMsg`: Message sent to all vertices in step 0.
- `vprog`: `(VertexId, VD, Msg) => VD` (Vertex update).
- `sendMsg`: `EdgeTriplet[VD, ED] => Iterator[(VertexId, Msg)]`.
- `mergeMsg`: `(Msg, Msg) => Msg` (Message combiner).

Pregel Type Signature

```
1 def pregel[A](initialMsg: A, maxIterations: Int, activeDirection: EdgeDirection)(  
2   vprog: (VertexId, VD, A) => VD,  
3   sendMsg: EdgeTriplet[VD, ED] => Iterator[(VertexId, A)],  
4   mergeMsg: (A, A) => A)  
5 : Graph[VD, ED]
```

Pregel Internal Sketch (Step 1-2)

1. Receive initial message at each vertex:

```
1  var g = mapVertices( (vid, vdata) => vprog(vid, vdata, initialMsg) ).cache()  
2
```

2. Compute initial messages:

```
1  var messages = GraphXUtils.mapReduceTriplets(g, sendMsg, mergeMsg)  
2
```

Pregel Internal Sketch (Loop)

1. While there are active messages and $i < \text{maxIterations}$:
 - Update vertices with received messages (`joinVertices`).
 - Compute new messages using `mapReduceTriplets` on edges where at least one endpoint received a message.
 - Cache new messages and uncache old ones.

Pregel Example: SSSP (Scala)

```
1 val initialGraph = graph.mapVertices((id, _) =>
2   if (id == sourceId) 0.0 else Double.PositiveInfinity)
3 val sssp = initialGraph.pregel(Double.PositiveInfinity)(
4   (id, dist, newDist) => math.min(dist, newDist), // vprog
5   triplet => { // sendMsg
6     if (triplet.srcAttr + triplet.attr < triplet.dstAttr) {
7       Iterator((triplet.dstId, triplet.srcAttr + triplet.attr))
8     } else Iterator.empty
9   },
10  (a, b) => math.min(a, b) // mergeMsg
11 )
```

Graph Builders

Ways to Build Graphs

- `GraphLoader.edgeListFile`: Load from a text file of edge pairs.
- `Graph.apply`: From vertex and edge RDDs.
- `Graph.fromEdges`: Automatically creates missing vertices.
- `Graph.fromEdgeTuples`: From RDD of (ID, ID) pairs.

Loading from Edge List

```
1 // File format: srcId dstId
2 // 1 2
3 // 2 3
4 val graph = GraphLoader.edgeListFile(sc, "data.txt")
```

All vertex and edge attributes default to 1.

Vertex and Edge RDDs

- Extends `RDD[(VertexId, VD)]`.
- Adds constraint: each `VertexId` occurs only once.
- Internally stores attributes in a reusable hash-map data structure.
- Joins between `VertexRDDs` can be done in constant time if they share the same index.

VertexRDD Methods

- `filter`: Preserves internal index.
- `mapValues`: Preserves internal index.
- `minus`: Shows vertices unique to the set.
- `diff`: Removes vertices present in another set.
- `leftJoin` / `innerJoin`: Optimized for shared indices.

- Extends `RDD[Edge[ED]]`.
- Organizes edges in blocks partitioned using `PartitionStrategy`.
- Methods: `mapValues`, `reverse`, `innerJoin`.

Optimized Representation

Vertex Cut vs. Edge Cut

- **Edge Cut:** Traditional approach. Cut edges to partition vertices.
- **Vertex Cut:** GraphX approach. Cut vertices to partition edges.
- Vertex cut reduces communication overhead for power-law graphs.

Graph Algorithms

- Measures importance of vertices.
- Vertex rank proportional to the probability that a random surfer lands on it.
- `graph.pageRank(tol)`: Runs until convergence.
- `graph.staticPageRank(numIter)`: Runs for fixed iterations.

Connected Components

- Labels each vertex with the ID of its lowest-numbered vertex in the same component.
- `graph.connectedComponents().vertices`
- Used to identify clusters in social networks.

Triangle Counting

- Counts triangles passing through each vertex.
- Requires edges to be in canonical orientation (`srcId < dstId`).
- Used to measure local clustering coefficients.

Comprehensive Example

Multi-step Analysis (1/2)

```
1 // Load edges and join with user attributes
2 val graph = followerGraph.outerJoinVertices(users) {
3   case (uid, deg, Some(attr)) => attr
4   case (uid, deg, None) => Array.empty[String]
5 }
6 // Restrict to users with specific attributes
7 val subgraph = graph.subgraph(vpred = (vid, attr) => attr.size == 2)
```

Multi-step Analysis (2/2)

```
1 // Run PageRank on subgraph
2 val pagerankGraph = subgraph.pageRank(0.001)
3 // Get attributes of top PageRank users
4 val topUsers = pagerankGraph.vertices.top(5)(Ordering.by(_._2))
```

Questions?